the three bottom-half mechanisms that we mostly deal with (in 2.6 series). The mechanisms are *softirqs*, *tasklets*, and *work queues*.

It is worth mentioning the *kernel timer* here itself. This mechanism essentially performs a postponement of the tasks for specific intervals of time. We will discuss its technical details in the coming days.

In this context, you may have a glance at the code portion that handles *kernel internal timers, kernel timekeeping* and *basic process system calls*. This is initiated by:

```
static DEFINE_PER_CPU(tvec_base_t, tvec_bases) = { SPIN_LOCK_
UNLOCKED };

static void check_timer_failed(struct timer_list *timer)
{
    static int whine_count;
    if (whine_count < 16) {
        whine_count++;
        printk("Uninitialised timer!\n");
        printk("This is just a warning.  Your computer is OK\n");
        printk("function=0x%p, data=0x%lx\n",
            timer->function, timer->data);
        dump_stack();
    }
    /*
     * Now fix it up
     */
    spin_lock_init(&timer->lock);
    timer->magic = TIMER_MAGIC;
}
```

And the 'starting' is done using:

```
void add_timer_on(struct timer_list *timer, int cpu)
{
    tvec_base_t *base = &per_cpu(tvec_bases, cpu);
    unsigned long flags;

    BUG_ON(timer_pending(timer) || !timer->function);

    check_timer(timer);

    spin_lock_irqsave(&base->lock, flags);
    internal_add_timer(base, timer);
    timer->base = base;
    spin_unlock_irqrestore(&base->lock, flags);
}
```

You can see the actual code that governs *softirqs* in *kernel/softirq.c. (*As I said, we rarely use *softirqs*. In most cases *tasklets* are employed and most of the drivers use *tasklets* for bottom half.) *softirqs* are represented using *softirq_action* which is defined as:

```
struct softirq_action {
    void (*action)(struct softirq_action *); /* function to run */
    void *our_data;                          /* data to pass to the function
*/
};
```

Correspondingly, a 32-entry array of this structure can be found in the above code file (*softirq.c*). Since one *softirq* needs one entry, there can be a maximum of 32 registered *softirq*s only. Also, you may see that the kernel actually uses only a fewer entries (out of this 32).

Here is a *softirq* handler for your reference:

```
void softirq_handler(struct softirq_action *)
```

The kernel uses a similar action function with a pointer to the respective *softirq_action* structure, when it runs the *softirq* handler. It is worth mentioning that the kernel passes the entire structure and this facilitates future additions to the structure without redoing the handler. The handler retrieves the data value by dereferencing the argument and looking for the data member. Also, you may find that a *softirq* never attempts to preempt another *softirq*, and the only way to preempt a *softirq* is by deploying an interrupt handler.

You need to check that the registered *softirq* is marked properly before executing it (technically termed as *raising the softirq*) and, normally, the handler marks the corresponding *softirq* for execution before returning.

The pending ones are executed in the following cases:

- Return from a hardware interrupt code
- *ksoftirqd* kernel thread
- Codes that look for pending *softirqs*

The execution occurs when *do_softirq()* is called. You can use this just by using the programming logic: if there are any pending ones, perform *do_softirq()* loop. Now let's look at this part of *do_softirq()*:

```
u32 pending = softirq_pending(cpu);

if (pending) {
  struct softirq_action *h = softirq_vec;

  softirq_pending(cpu) = 0;

  do {
    if (pending & 1)
      h->action(h);
    h++;
```